

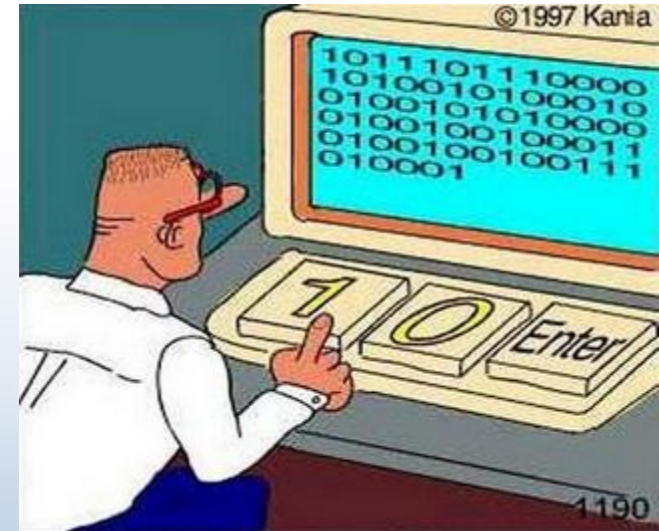
Qualidade para Escrita de Software

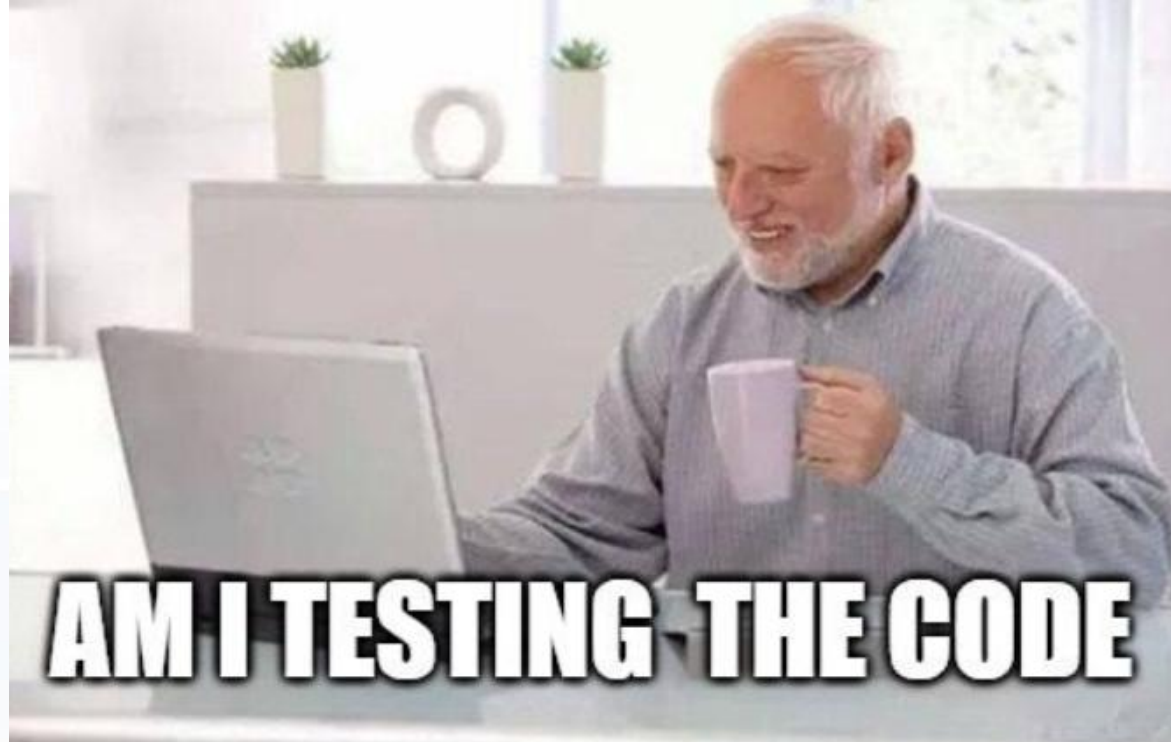
Prof. Dr. Thiago Jabur Bittar

Universidade Federal de Catalão

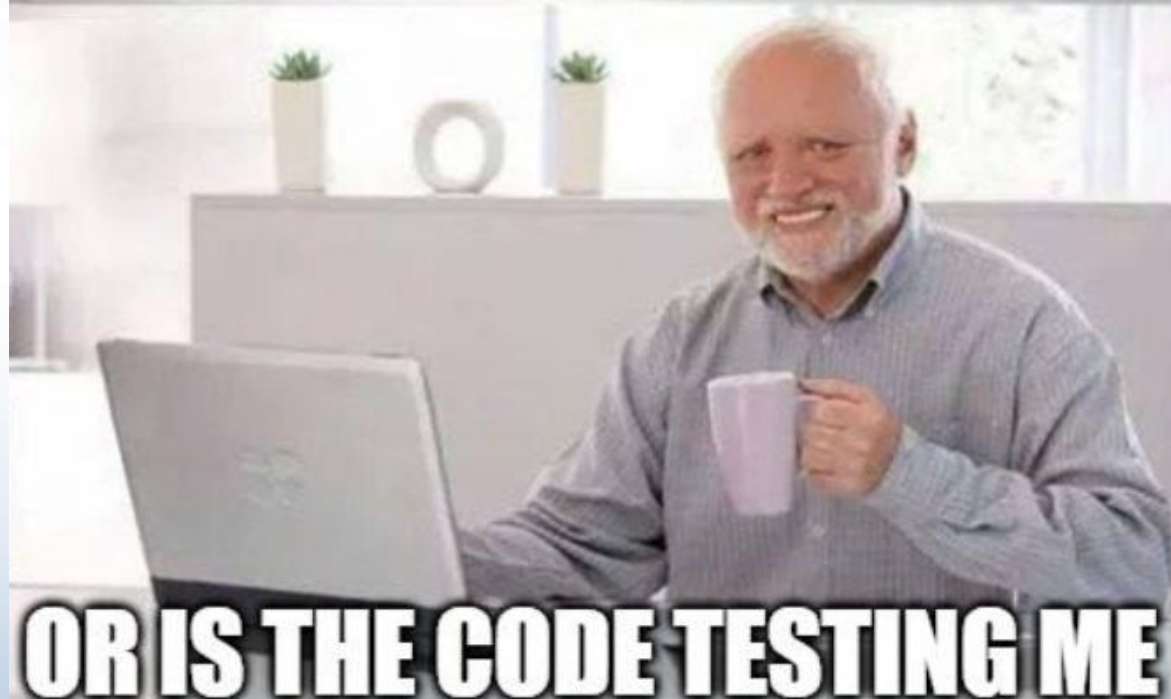
Código com qualidade

- Os códigos:
 - Devem ser feitos para serem lidos por seres humanos!
- Escreva os comentários no momento em que estiver escrevendo o algoritmo.



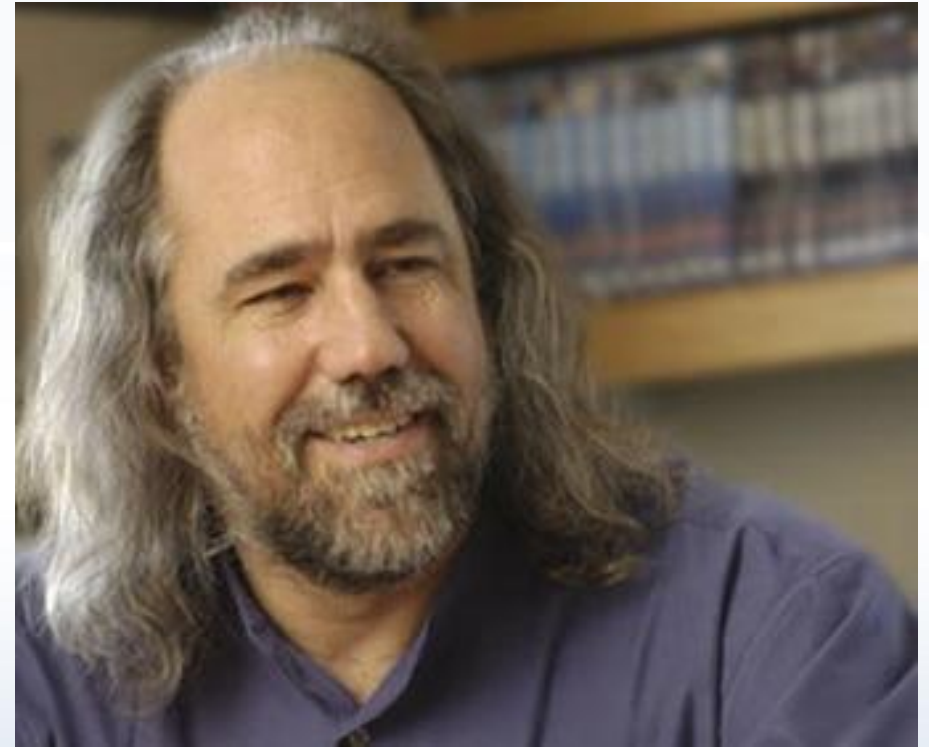


AM I TESTING THE CODE



OR IS THE CODE TESTING ME

- “Código limpo é simples e direto. Código limpo é lido como uma prosa bem escrita”.
 - Grady Booch,
 - um dos criadores da UML



Características de um código de qualidade

- Legibilidade: O código deve ser fácil de ler e entender, com uma sintaxe clara, sem abreviações confusas e com um estilo de codificação consistente;
- Simplicidade: conciso para reduzir erros e bugs;
- Modularidade: O código precisa ser dividido em módulos independentes, cada um com uma responsabilidade clara e bem definida. Isso facilita o teste, a depuração e a manutenção do código;

Características de um código de qualidade

- Documentação: ajuda outros membros da equipe a entender a função de cada parte do código;
- Testabilidade: Deve ser fácil de testar e possuir um conjunto abrangente de testes unitários para garantir seu funcionamento correto;
- Confiabilidade: Deve ser confiável e lidar adequadamente com erros e exceções, retornando mensagens de erro claras e informativas quando ocorrerem.

Características de um código de qualidade

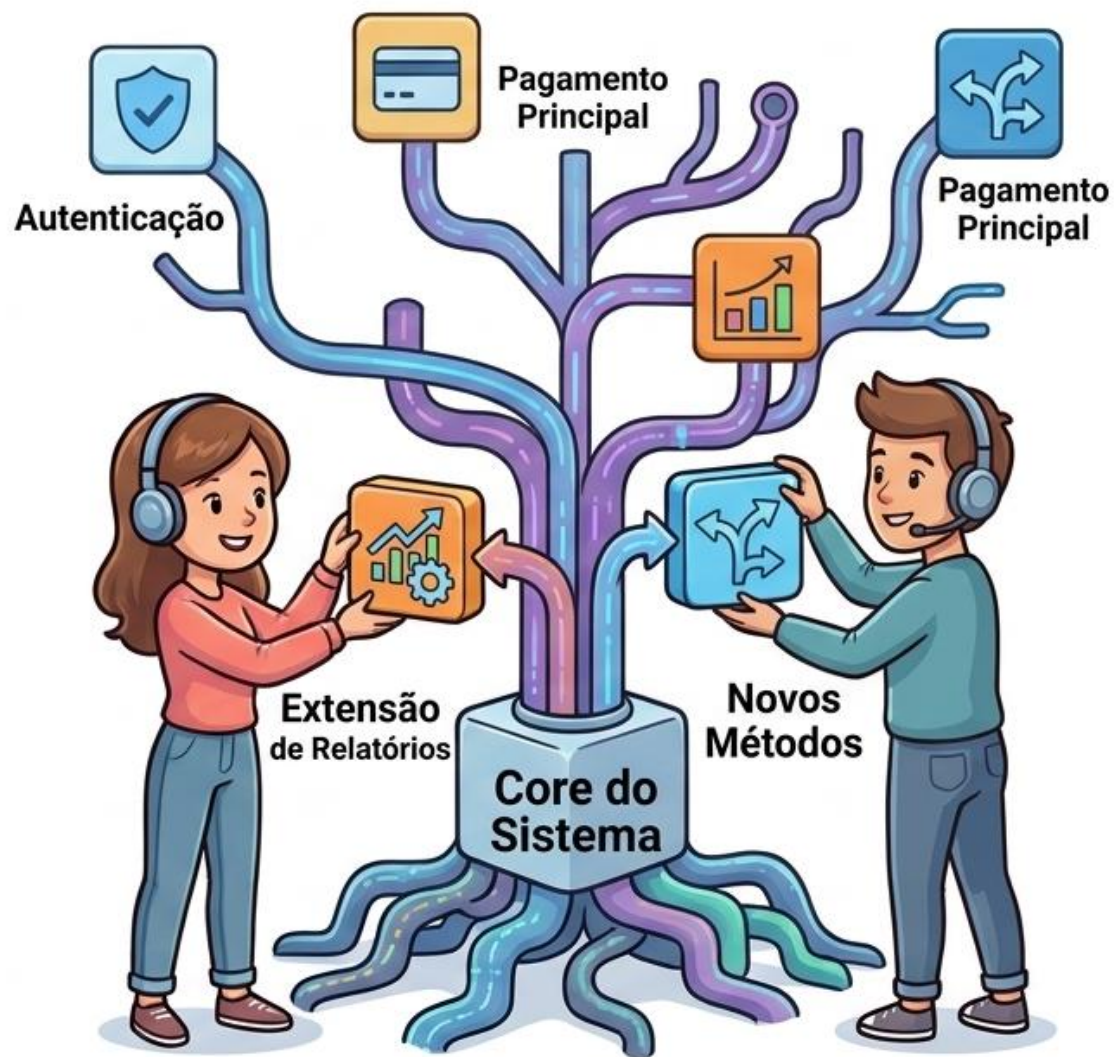
- Fácil de estender
- Reusável no futuro



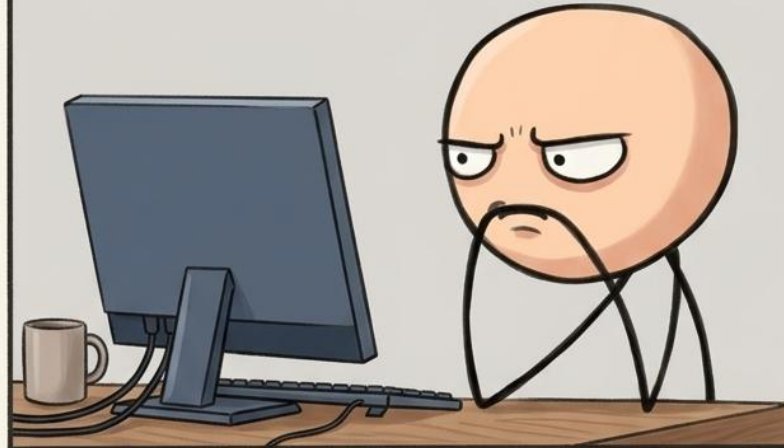
REUSABILIDADE DE SOFTWARE



EXTENSIBILIDADE DE SOFTWARE



Não funciona..... por quê?



Funciona..... por quê?

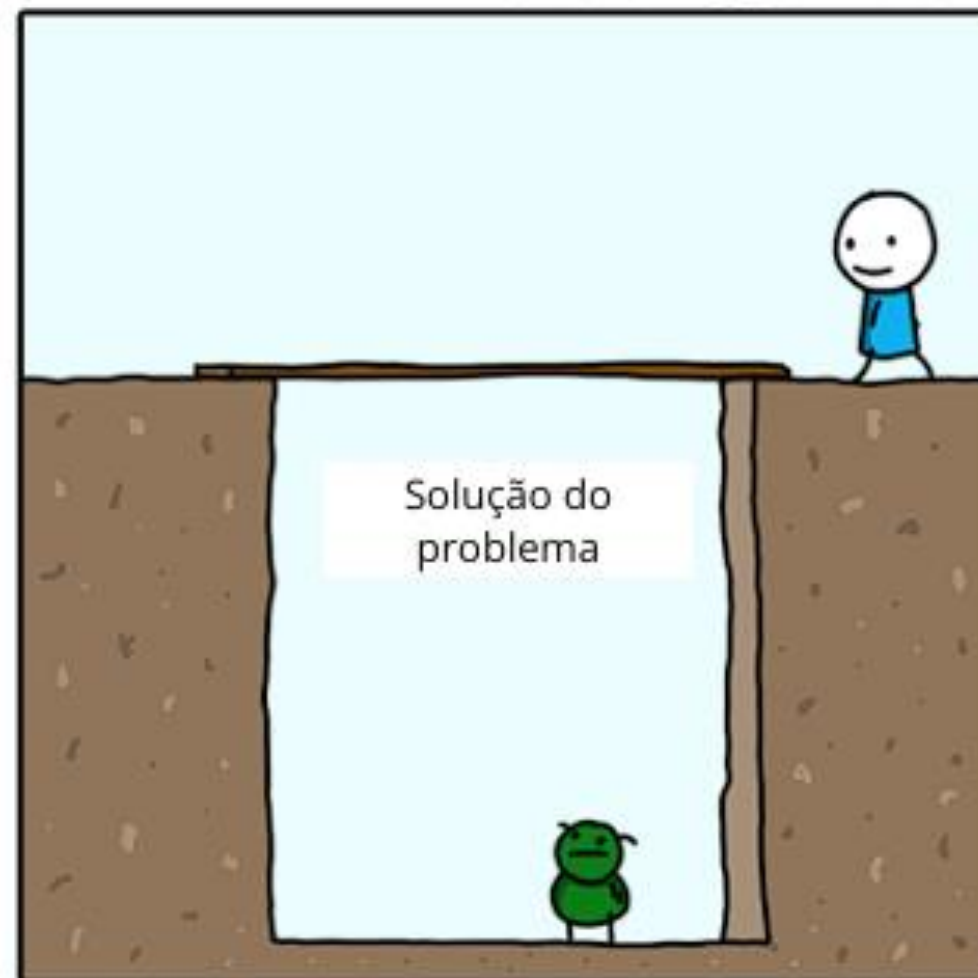
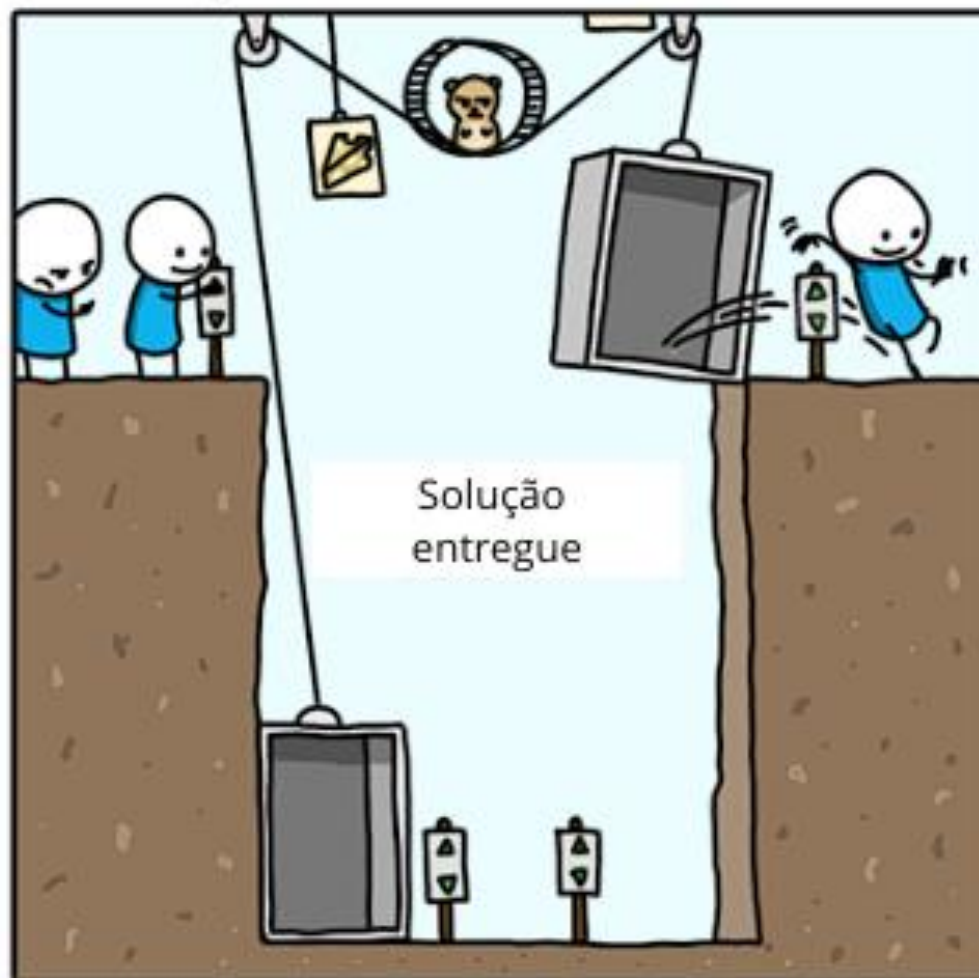


Princípio KISS

- Mantenha as coisas simples!
 - Normalmente tendemos a complicar as coisas que poderiam ser muito mais simples.
- Então, Keep It Stupid Simple
 - Mantenha isto estupidamente simples - KISS!

Complexidade

MONKEYUSER.COM

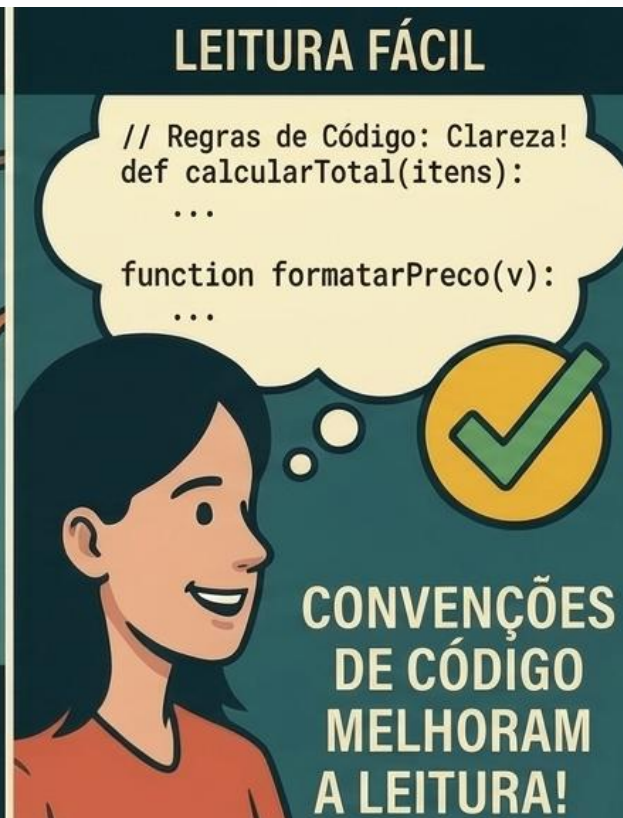


Motivação

- Quem escreveu não deve ser o único a poder ler o código
- Criar um padrão (consensual)
 - Em parte independente da linguagem

Mais razões

- 80% do tempo de desenvolvimento é para manutenção;
- Quase nenhum software é mantido apenas pelo primeiro desenvolvedor;
- **Convenções de código** melhoram a legibilidade do código;
- Se o código fonte é fornecido com o sistema, ele deve estar bem apresentado.



```
int main(int argc, char *argv[]){float a,b,c;cout<<
Entre com três valores reais..."<<endl;cin>>a>>b>>c;
float m=(a+b+c)/2;cout<<"Média: "<<m<<endl;return 0;}
```



Funciona, porém
está **ilegível!**

```
int main(int argc, char *argv[])
{
```

```
    float a, b, c;
```

```
    cout << "Entre com três valores reais..." << endl;
```

```
    cin >> a >> b >> c;
```

```
    float m = (a + b + c) / 2;
```

```
    cout << "Média: " << m << endl;
```

```
    return 0;
```

```
}
```



Funciona e está
legível!

Algoritmos com qualidade

- Os comentários devem acrescentar alguma coisa, e não frasear o comando:

```
// Multiplicação de b por h:  
area ← b * h;
```



```
// Cálculo da área do retângulo:  
area ← b * h;
```

Algoritmos com qualidade

- Use espaços em branco para melhorar a legibilidade:

```
hip←sqrt(cat1*cat1+cat2*cat2) ;
```



```
hip ← sqrt(cat1 * cat1 + cat2 * cat2) ;
```

Algoritmos com qualidade

- Escolha nomes **representativos** para as variáveis:

```
p ← c + 1;
```



```
preco ← custo + lucro;
```

```
//abreviação obscura
```

```
const h = 8760
```

```
for (i=0; i < h; i++) {
```

```
}
```

```
//forma descritiva
```

```
const totalHoursInAYear = 8760
```

```
for (i=0; i < totalHoursInYear; i++) {
```

```
}
```

Algoritmos com qualidade

- Utilize um comando por linha
- Utilize parênteses para melhorar a compreensão e evitar erros
- Utilize indentação (recuo de texto)

Comentários linhas gerais

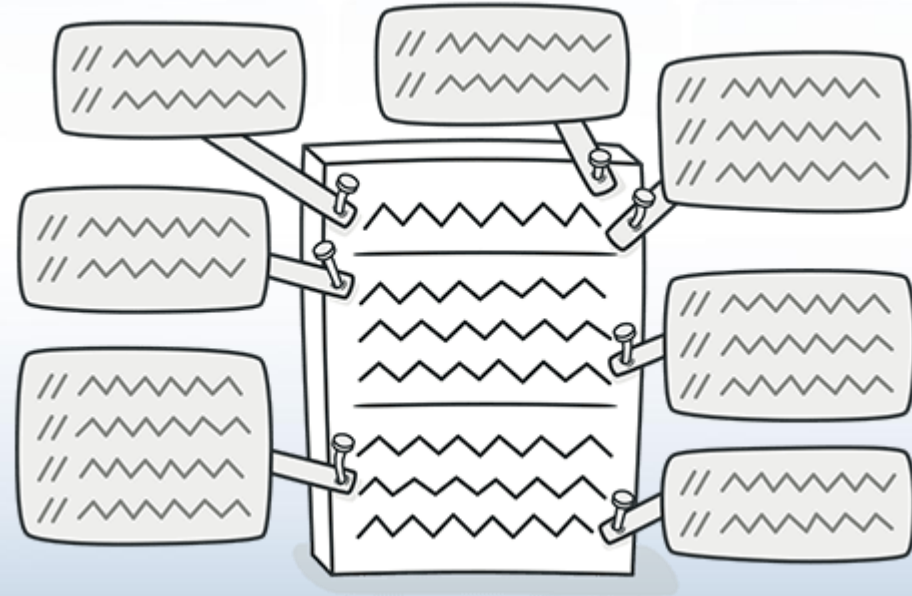
- Dois tipos de comentários (da implementação e da documentação)
- Linguagens modernas tem formas específicas
 - `//`
 - `/* */`
- Não adicione comentários redundantes
 - O excesso de comentários reflete código mal escrito
 - pense em rescrever o trecho...

Comentários

- Comentários muito curtos podem estar na mesma linha
- Se existem vários os mesmos devem estar alinhados em blocos

Cuidado com comentários

- Comentários são frequentemente usados como desodorante para cobrir o cheiro do código que poderia ser melhorado
- Se houver muitos comentários, significa que o código é ruim ou não é fácil de entender



- “Quando você sentir necessidade de escrever um comentário, primeiro tente refatorar o código para que qualquer comentário se torne supérfluo”
- Comentários errados ou desatualizados são piores que nenhum comentário

Comandos

- Colocar um comando por linha;
- Em um bloco indentar sempre;
- Colocar { no final da linha onde começa o bloco e } alinhado com o início desta.;
- Use {} mesmo quando os blocos tiverem apenas um comando.

Nome dos Arquivos

- Mesmo que a linguagem não exija, use sufixos claros
 - ex. .java, .c, .cpp, .txt
- Sempre tenha no diretório um ***README***
 - este arquivo deve conter um resumo do conteúdo do diretório (pasta)

Dicas de programação

Use parênteses

```
if (a == b && c == d)          //evitar
```

```
if ((a == b) && (c == d)) //ok
```

Use operadores ternários (em certas situações)

```
if (cond) {  
    return x;  
}  
return y;
```

```
return (cond ? x : y);
```

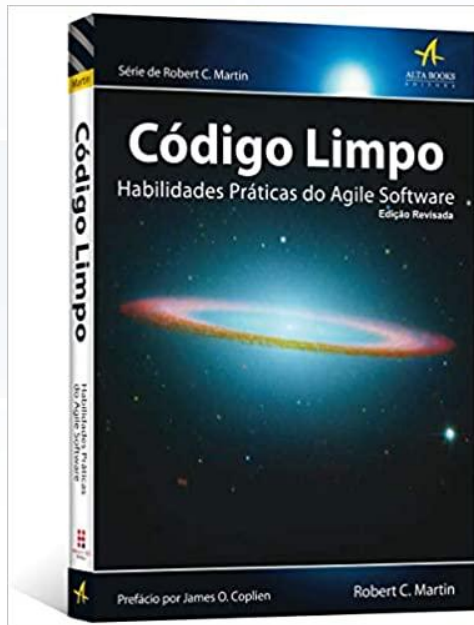
Coloque o comentário XXX se há algo com problema e
FIXME onde existem *bugs*

Utilize DRY

- DRY é o acrônimo para *Don't repeat yourself* (Não repita a si mesmo). É o conceito que diz que cada parte de conhecimento do sistema deve possuir apenas uma representação.
- Evite a ambiguidade do código. Em outras palavras, não deve existir duas partes do programa que desempenham a mesma função, ou seja, **o famoso copiar e colar no código**.

Código Limpo - Clean code

8 setembro 2009 - 1ª edição



- Mesmo um código ruim pode funcionar.
- **Mas se ele não for limpo, pode acabar com uma empresa de desenvolvimento.**
- Perdem-se a cada ano horas incontáveis e recursos importantes devido a um código mal escrito. Mas não precisa ser assim.
- **R\$ 79,90**

Trabalho Eficaz com Código Legado (Português)

– 5 abril 2013 – 3ª edição



- Seu código é fácil de alterar? Você recebe feedback quase instantâneo quando o altera? Você o entende?
- Se a resposta a alguma dessas perguntas for “não”, você tem um código legado e ele está consumindo tempo e dinheiro de seus esforços de desenvolvimento.
- **R\$ 96,85**

Clean Code

- Ou “Código Limpo”
- Termo popularizado por Uncle Bob (Robert C. Martin) que define uma série de práticas destinadas
 - a melhorar a qualidade do código de programação
- Legibilidade, simplicidade e eficiência do código

Evite Números Mágicos

- Números mágicos são valores numéricos com significado não claro no código
 - Substitua-os por constantes nomeadas para tornar o código mais legível.
- Por exemplo, substitua 86400 (número de segundos em um dia) por `SEGUNDOS_POR_DIA`.

Funções Pequenas e Específicas

- Mantenha suas funções curtas e focadas em uma única responsabilidade
 - Isso torna mais fácil entender o que a função faz e ajuda na reutilização do código.
 - Evite funções longas que fazem várias coisas.

Bad Smell

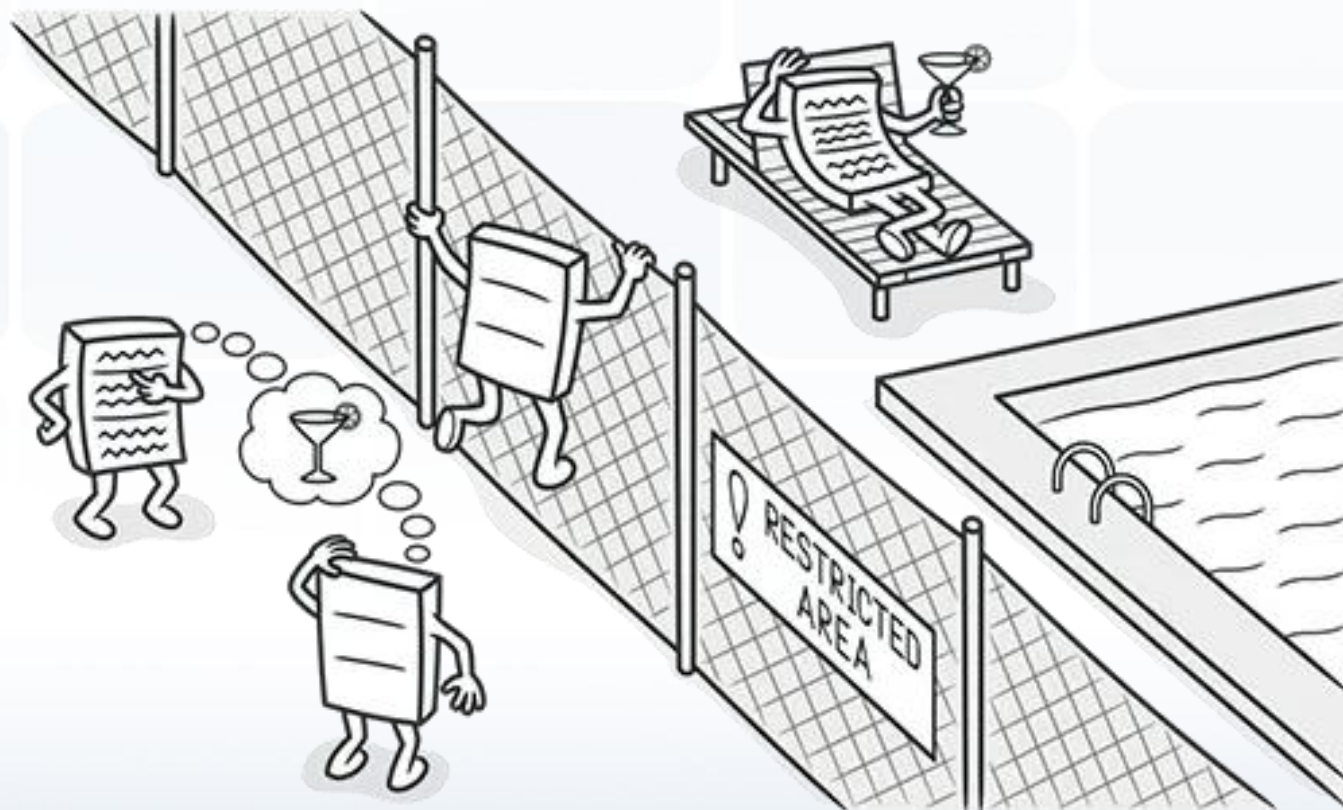
- Acontece quando o código é desenvolvido de uma forma pobre em seu design ou quando a implementação escolhida foi errada
 - Martin Fowler
- Code Smells não são bugs, contudo, eles indicam pontos fracos que podem ocasionar falhas no presente ou no futuro.



Bad Smells (mal cheiro) – Falta de coesão

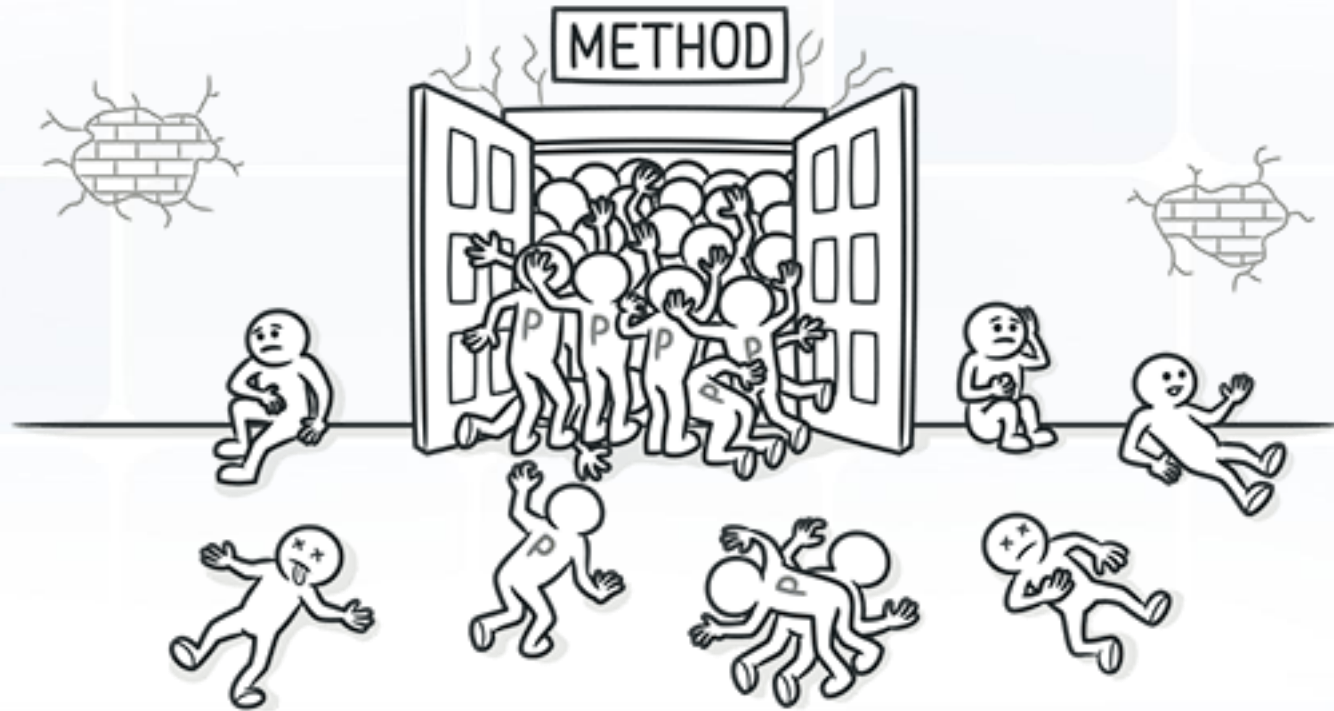
- Código Duplicado
- Método Longo
- Classes Grandes
- Feature envy (inveja de recursos)
 - uma classe que utiliza em excesso métodos de outra classe
- Classe preguiçosa
 - classe que faz muito pouco

Feature envy



Bad Smells (mal cheiro) – Falta de coesão

- **Lista de Parâmetros Longa**
- Quando você está criando uma função e ela requer alguma entrada, é melhor passá-la nos parâmetros da função em vez de usar dados globais
- Mas às vezes aumentamos o número de parâmetros e, muitas vezes, ficamos confusos



Lista de Parâmetros Longa

- Solução

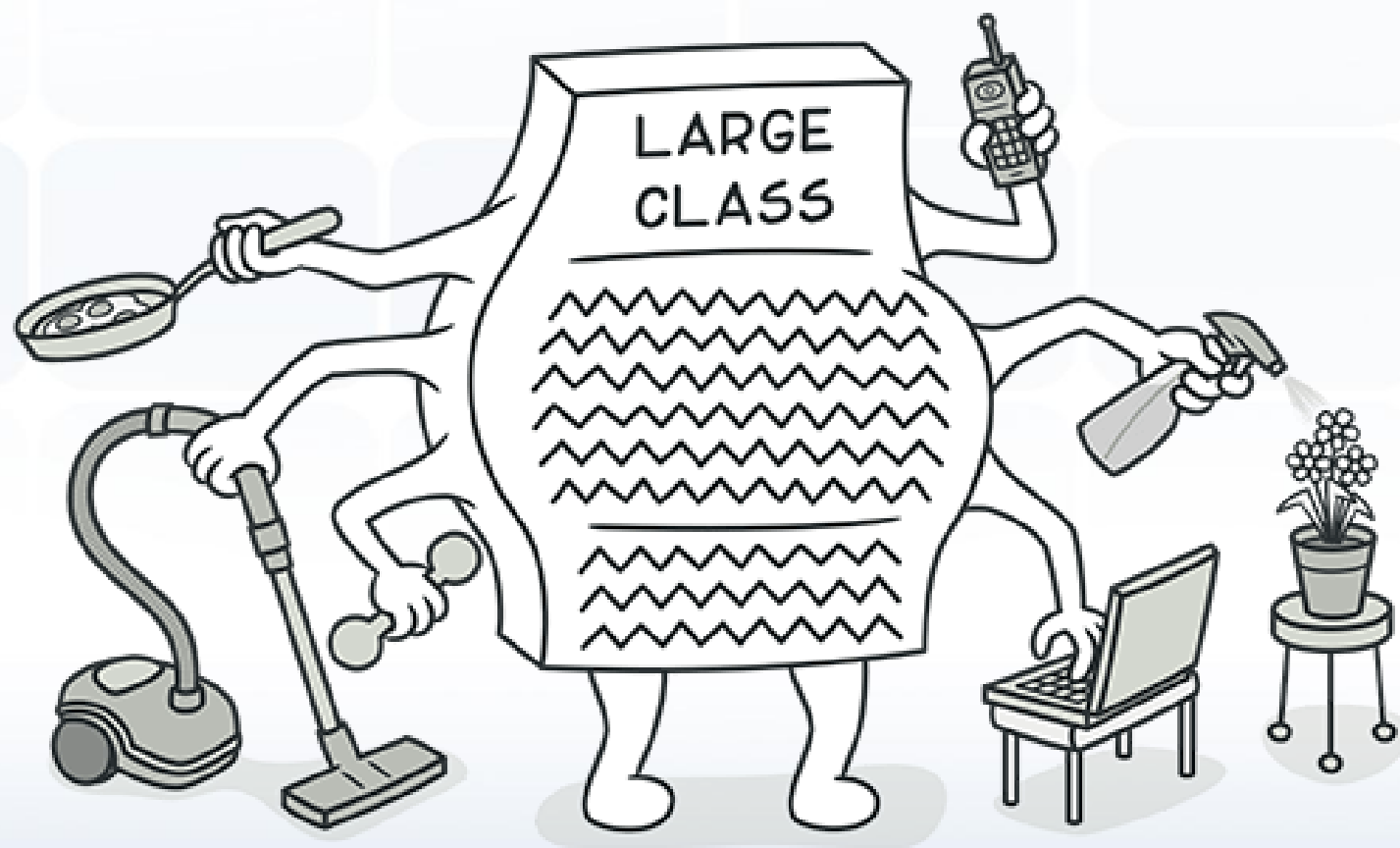
- Substituir Parâmetros por Consulta: Chamar um método de consulta e obter resultados, então passar esses resultados como parâmetros para outra função.

```
//problem
```

```
int basePrice = quantity * itemPrice;  
double seasonDiscount = this.getSeasonalDiscount();  
double fees = this.getFees();  
double finalPrice = discountedPrice(basePrice,  
seasonDiscount, fees);
```

```
//solution
```

```
int basePrice = quantity * itemPrice;  
double finalPrice = discountedPrice(basePrice);
```

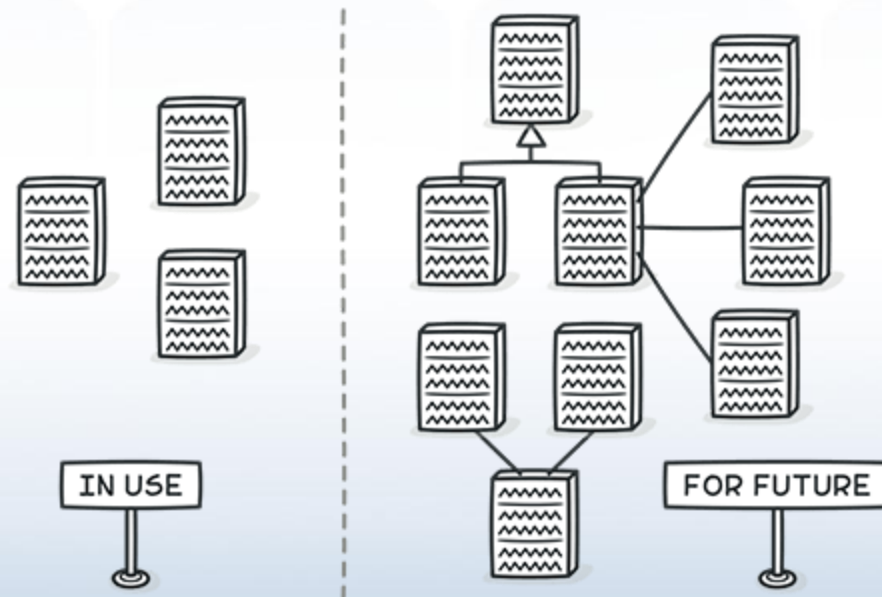


Bad Smells (mal cheiro) – Falta de coesão

- Complexidade artificial
 - uso forçado de design patterns complicados
 - onde um design simples seria suficiente

Código em excesso

- Quando estamos escrevendo código, adicionamos alguma funcionalidade extra ou escrevemos algum código extra para suportar recursos futuros que nunca são implementados.
- Como resultado, nosso código se torna difícil de entender e manter



Refatorar

- Refatorar é melhorar o código depois de ele ter sido escrito
- "Refatoramento é o processo de mudança de um sistema de software de tal maneira que não altere o comportamento externo do código e **ainda melhore a sua estrutura interna**"